Supplier Search Agent

AI-powered supplier discovery and contact extraction platform for construction procurement workflows

Andrea Bochicchio & Filippo Gambelli

Artificial Intelligence and Data Engineering, DII, Unipi

June 21, 2026

Project Source Code:
github.com/FilippoGambelli/supplier-search-agent

Agentic AI for Supplier Discovery

The Problem

Finding new suppliers is still a highly manual and time-consuming process. Procurement teams need to:

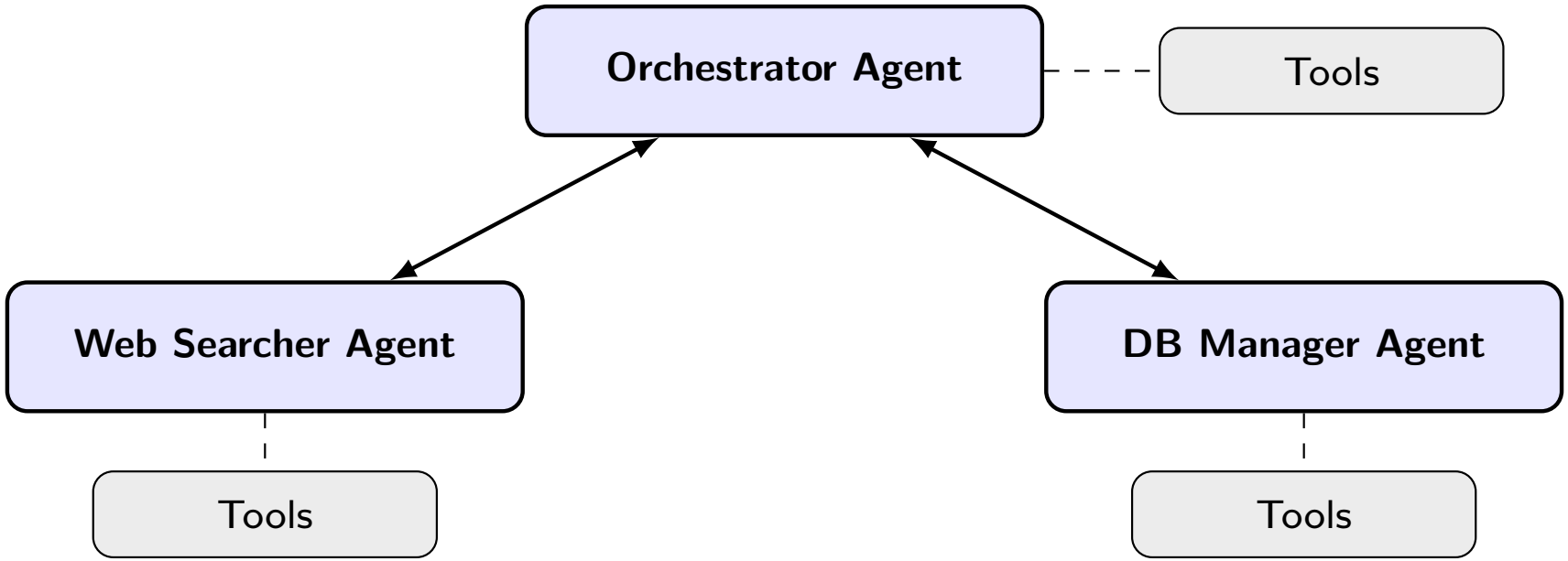
- Search for potential suppliers across multiple online sources
- Gather company information and contact details
- Organize the collected information
- Repeat the same work every time a new sourcing request arises

Our Solution

Supplier Search Agent $\rightarrow$ a multi-agent AI system that automates the entire supplier sourcing process. Users provide a natural language request, and the system:

- Searches for relevant suppliers within the internal company database
- Performs live web discovery to find new relevant suppliers and updates the database accordingly
- Returns structured company information and contact details

Multi-Agent Architecture



The platform is built on **LangChain** and **LangGraph** to support structured LLM workflows, tool execution, and multi-agent orchestration.

Orchestrator Agent

Orchestrator Role

The Orchestrator is the brain of the system. It receives the user request and decides how to execute the supplier search workflow.

Responsibilities:

- **Understanding** → interprets procurement requirements
- **Coordination** → activates Web Searcher and DB Manager agents
- **Returns** → structures the final supplier list

Workflow

1. The Orchestrator forwards the request to the Web Searcher Agent
2. The Web Searcher retrieves supplier information from the web and returns structured results
3. The Orchestrator sends the retrieved data to the DB Manager Agent
4. The DB Manager checks existing records to update them if information is missing or outdated, or adds them as new entries if they are not yet in the database
5. The Orchestrator queries the DB Manager to retrieve the final supplier set

Web Searcher Agent (1)

Web Searcher Agent discovers new suppliers on the web based on a natural language query. It exposes a set of native tools:

Main functionalities

- **Search Suppliers:** Queries the SearXNG instance to retrieve candidate URLs containing titles and links
- **Extract from PagineGialle:** A dedicated crawler that extracts real company websites from PagineGialle directory pages
- **Extract:** Scrapes the HTML of the Homepage and Contact page (respecting `robots.txt`), cleans the text, and invokes the LLM for structured data extraction

The Web Searcher Agent supports two alternative execution strategies:

- **Deterministic Workflow**
- **Agentic Workflow**

Web Searcher Agent - Deterministic Workflow (2)

Our initial approach was to build a deterministic workflow using LangGraph, with predefined steps and nodes, leveraging an LLM exclusively to extract company information

LangGraph Node Flow

The logical steps are hardcoded into the graph using explicit conditional edges:

1. **search:** Executes the web search using the SearXNG instance
2. **extract_pg:** Dedicated node to handle PagineGialle links and extract real company URLs
3. **scrape:** Combines results from `search` and `extract_pg`, applies hostname-based deduplication and handles website scraping
4. **extract $\leftrightarrow$ llm loop:** The system uses the `extract` node to select the current company; it routes to the `llm` node for data extraction
5. **final_answer:** Returns aggregated results

Web Searcher Agent - Agentic Workflow (3)

This implementation relies entirely on the reasoning and planning capabilities of the LLM. The behavior of the agent is bounded by a structured **System Prompt** that enforces a sequential 6-step execution:

Agentic Workflow

- **Search:** Invokes `search_suppliers` using the original query.
- **Validation:** Evaluates each candidate URL using `is_valid_company`. If the URL belongs to PagineGialle, it triggers the `PagineGialle extraction` process; otherwise, it verifies that the website is associated with a specific company and excludes aggregator platforms or other non-company websites.
- **PagineGialle Extraction:** Extracts actual company website links from identified PagineGialle pages
- **Deduplication:** Removes duplicate URLs by comparing only their base hostnames
- **Extraction:** Performs deep data extraction across the entire cleaned final list
- **Final Response:** Outputs a single structured JSON array

Core Characteristic

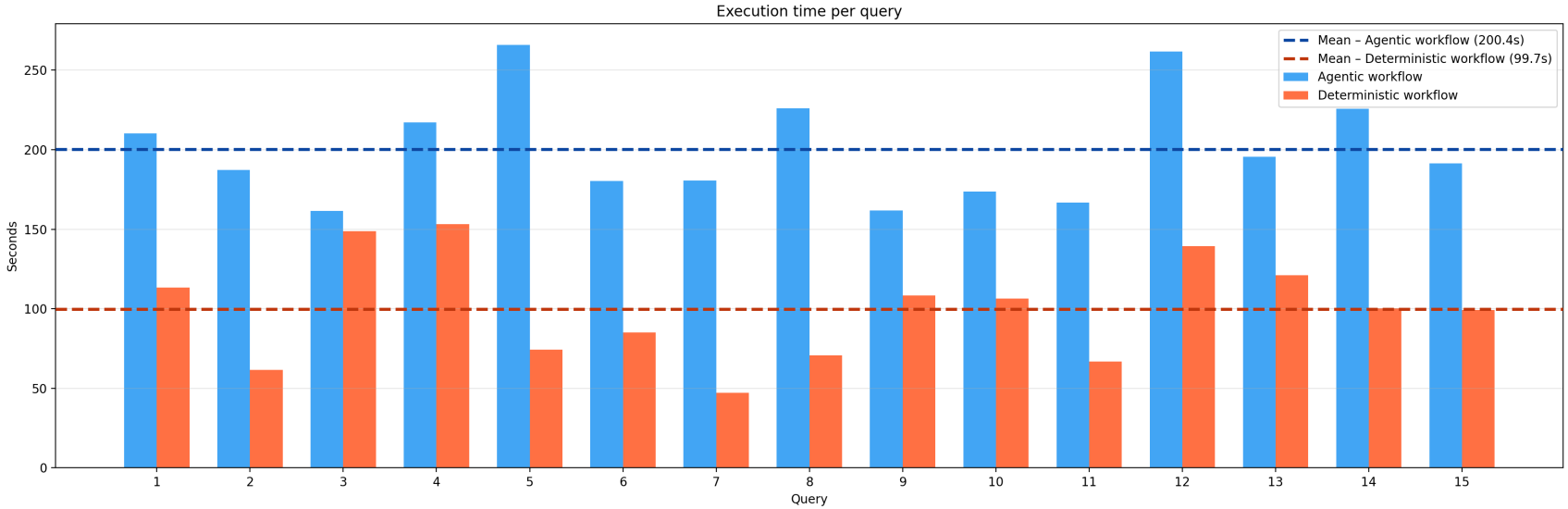
Using an agentic approach, the LLM **dynamically** selects the next tool based on previous outputs, improving both the search and decision-making phases by identifying the most relevant source for extraction and avoiding low-value aggregator websites.

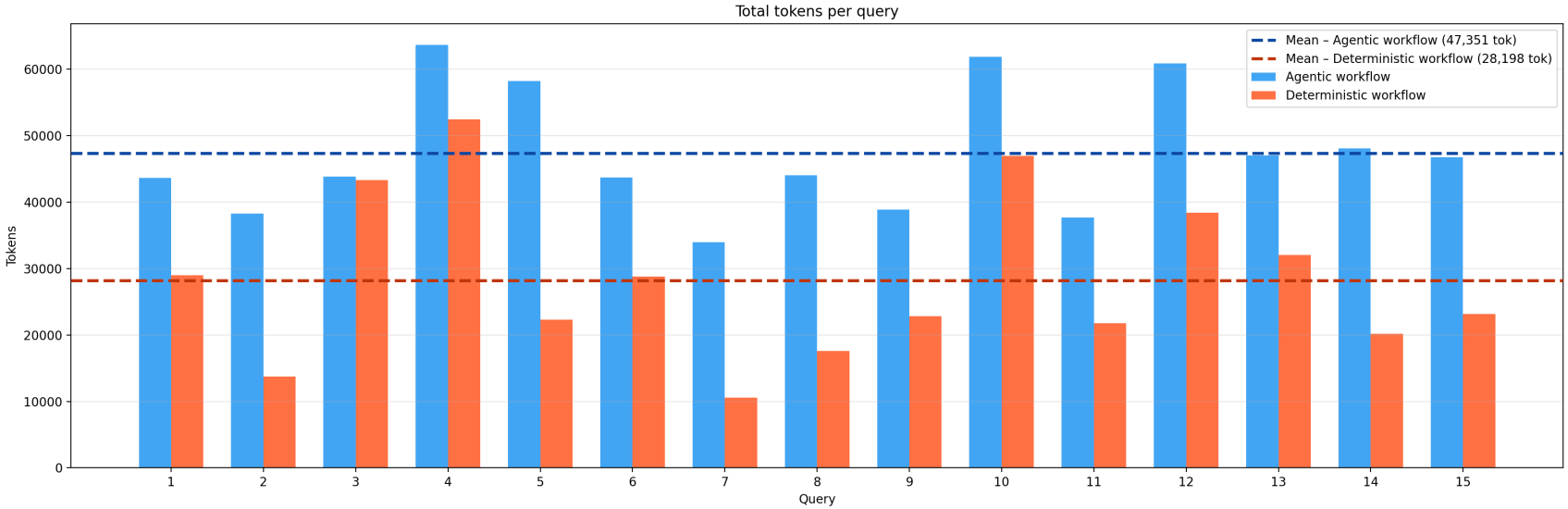
Web Searcher Agent - Strategy Comparison (4)

The platform provides both approaches:

Feature	Agentic Web Search	Deterministic Pipeline
Flow Control	Dynamic (LLM)	Rigid (Graph-Coded)
Speed / Latency	Variable (Multiple LLM calls)	More efficient and linear
Debugging	Complex (ReAct tracing)	Simple (Single node inspection)
Adaptability	High on heterogeneous data	Limited to predefined rules
Failsafe / Errors	Handled autonomously	Defined by fallback nodes

Web Searcher Agent - Experimental Results (5)





Test setup: top 15 web results were used; if PagineGialle appeared, only the first 5 companies were extracted. The extraction LLM ran without reasoning.

Database Manager Agent (1)

Database Manager Agent role

The Database Manager handles all persistence and retrieval operations, maintaining a continuously growing supplier knowledge base.

DB Manager functions:

`save_supplier_to_db`

- **Data Cleaning:** Runs domain-specific normalization on websites, arrays of emails and phones
- **Check for Duplications:** Checks for duplicates to detect overlapping elements in VAT or contact array; if no match is found, a new entry is created
- **Dynamic Update:** If a match is found, it updates properties or appends missing locations
- **Embedding Refresh:** Concatenates description and updated categories, triggering a vector model recalculation for future semantic searches

`execute_search_query`

- **Hierarchical Filtering:** Generates a flexible raw SQL query that appends pattern-matching clauses dynamically based on the provided geographic levels
- **Semantic Ranking:** Converts the user's search text into a vector embedding and injects it into the SQL query
- **Cosine Distance Operator:** Utilizes `pgvector`'s native `<=>` operator to calculate the distance between the query vector and the suppliers' stored embeddings

Database Manager Agent (2)

Supplier Table

- Name
- Website
- Description
- Category
- Email
- Phone
- VAT number
- Embedding (vector 384-dim)

Purpose: Stores core supplier information used for search, matching, and retrieval.

SupplierLocation Table

- Country
- Region
- Province
- City
- Address

Purpose: Stores one or more physical locations associated with each supplier.

Relationship: Each **Supplier** can have multiple **SupplierLocation** entries (one-to-many relationship).

Project Particularities - Artifacts

Artifacts Pattern

A key architectural decision: data is passed between agents via **Artifacts** rather than through the LLM context window.

Why artifacts?

- Avoids context window overflow with large scraped pages
- Provides structured, type-safe data transfer between nodes
- Cleaner separation of concerns between agents
- Reduces LLM token consumption and cost
- Prevents agents from miscopying or corrupting information across context and response transitions

Implementation:

- `Artifact` objects carry supplier data between Orchestrator, Web Searcher, and DB Manager
- Each agent reads/writes artifacts without polluting its prompt context

Project Particularities - Category Management

From Unstructured Text to Structured Data

During the *Extraction* phase, the system uses an LLM to convert raw website text into structured information, extracting not only contacts and general company details, but also generating a set of semantic categories.

Workflow:

- **LLM-Driven Extraction:** The LLM identifies what the company *does, sells, or produces*, returning short noun phrases (e.g., *"industrial automation"*).
- **Persistent Storage:** The extracted category fields are persistently stored in the database, enabling future queries that combine location-based filtering with semantic search, allowing a ranking of companies based on their relevance to the user's request.

Output Example

Input: "Specialisti in sistemi di aria condizionata e HVAC"
Output: `["climatizzazione", "impianti hvac", "condizionamento"]`

Technology Stack & Infrastructure

AI & Inference Framework

- **Engine:** Ollama v0.24.0
- **Model:** `qwen3.6:35b`
- **Hardware:** NVIDIA GH200 480GB, CUDA 12.3

Inference Performance

$\sim$ **100 tokens/s**

Backend & Search Infrastructure

- **Database:** PostgreSQL with the `pgvector` extension for persistent data storage and semantic similarity matching.
- **Web Search:** SearXNG metasearch engine to perform real-time, rate-limit-free supplier discovery on the web.

Summary and Future Work

What We Built

A production-ready multi-agent system for automated supplier discovery that:

- Accumulates company information over time via persistent vector DB
- Handles real-world scraping challenges (robots.txt, language, categories)
- Combines agentic AI with traditional pipelines
- Provides both agentic (flexible) and deterministic (predictable) modes

Future Work:

- Currently, the system accepts a single simple query. However, users may request multiple companies or companies across different locations. In these cases, a query decomposition step could be introduced to split the input into multiple queries
- Allow users to directly insert company data they already know into the internal database, enabling manual enrichment and faster onboarding of trusted suppliers
- Provide the option for users to choose the search scope: internal database only, or a hybrid approach combining both sources (as implemented in the current system)